

Chapter Three

Introduction to object-oriented programming

3.1. Class and Object

The main purpose of C++ programming is to add object orientation to the C programming language and classes are the central feature of C++ that supports object-oriented programming.

C programming language is purely structured / procedural oriented while C++ can support both the structured / procedural oriented and object oriented approach.

A class is

- A user defined data type
- Used to specify and combine data representation/state and methods/functions for manipulating that data into one well-ordered package.
- A blue print for instantiating objects of that class.

An object is a single instance of a class that can access all members of the class

The data and functions within a class are called members of the class.

3.2. C++ Class Definitions

When we define a class, we define a blueprint for a user defined data type. This doesn't actually define any data, but it does define what the class name means, that is, what states an object of the class will consist of and what operations can be performed on such an object.

A class definition starts with the keyword `class` followed by the class name; and the class body, enclosed by a pair of curly braces. A class definition must be followed either by a semicolon or a list of declarations.

For example, we can define the Box data type using the keyword `class` as follows:

```
class Box
{
private:
...
protected:
...
public:
double length; // Length of a box
double breadth; // Breadth of a box
double height; // Height of a box
};
```

The keyword `public` determines the access attributes of the members of the class that follows it. A public member can be accessed from outside the class anywhere within the

scope of the class object. We can also specify the members of a class as private or protected which will be discussed in a sub-section.

3.3. Define C++ Objects

A class provides the blueprints for objects, so basically an object is created from a class. We declare objects of a class with exactly the same sort of declaration that we declare variables of basic types like int, float, double etc. Following statements declare two objects of class Box:

```
Box Box1; // Declare Box1 of type Box
```

```
Box Box2; // Declare Box2 of type Box
```

Both of the objects Box1 and Box2 will have their own copy of data members.

3.4. Accessing the Data Members

The public data members of objects of a class can be accessed using the direct member access operator (.. The following example can make the things clear:

```
#include <iostream>
using namespace std;
class Box
{
public:
double length; // Length of a box
double breadth; // Breadth of a box
double height; // Height of a box
};
int main( )
{
Box Box1; // Declare Box1 object of type Box
Box Box2; // Declare Box2 object of type Box
double volume = 0.0; // Store the volume of a box here
// box 1 specification
Box1.height = 5.0; // dot operator used
Box1.length = 6.0;
Box1.breadth = 7.0;
// box 2 specification
Box2.height = 10.0;
Box2.length = 12.0;
Box2.breadth = 13.0;
// volume of box 1
volume = Box1.height * Box1.length * Box1.breadth;
cout << "Volume of Box1 : " << volume << endl;
// volume of box 2
volume = Box2.height * Box2.length * Box2.breadth;
cout << "Volume of Box2 : " << volume << endl;
```

```
    return 0;
}
```

When the above code is compiled and executed, it produces the following result:



```
C:\Users\ADDISU\Documents\obj.exe
Volume of Box1 : 210
Volume of Box2 : 1560
```

It is important to note that private and protected members cannot be accessed directly using direct member access operator (.. We will learn how private and protected members can be accessed.

3.5. Classes & Objects in Detail

So far, we have got very basic idea about C++ Classes and Objects. There are further interesting concepts related to C++ Classes and Objects which we will discuss in various sub-sections listed below:

3.6. Concept Description

Class member functions:- A member function of a class is a function that has its definition or its prototype within the class definition like any other variable. It operates on any object of the class of which it is a member, and has access to all the members of a class for that object.

Class access modifiers:- A class member can be defined as public, private or protected. By default members would be assumed as private.

3.7. Constructor & destructor

A class constructor is a special function in a class that is called when a new object of the class is created. A constructor has no return type and its name is similar to that of the class name. Class constructors can be overloaded. A destructor is also a special function which is called when created object is deleted.

```
Box myBox; // Create an object
myBox.getVolume(); // Call member function for the object
#include <iostream>
using namespace std;
class CRectangle {
int width, height;
public:
```

```
CRectangle (); // default constructor
CRectangle (int,int); //parameterized constructor
int area (void) {return (width*height);}
};
CRectangle::CRectangle () {
width = 5;
height = 5;
}
CRectangle::CRectangle (int a, int b) {
width = a;
height = b;
}
int main () {
CRectangle rect (3,4);
CRectangle rectb;
cout << "rect area: " << rect.area() << endl;
cout << "rectb area: " << rectb.area() << endl;
return 0;}
```

3.8. SCOPE RESOLUTION OPERATORS

This approach is used to define outside of a class boundary. Member functions can be defined within the class definition or separately using scope resolution operator, (::). Defining a member function within the class definition declares the function inline, even if you do not use the inline specifier. So either we can define Volume() function as below: We can define the same function outside the class using the scope resolution operator (::) as follows:

```
double Box::getVolume(void)
{
    return length * breadth * height;
}
```

Here, only important point is that you would have to use class name just before (::) operator. A member function will be called using a dot operator (.) on an object where it will manipulate data related to that object only as follows:

```
//full implementation is demonstrated as below
#include <iostream>
using namespace std;
class Box
{
public:
double length; // Length of a box
double breadth; // Breadth of a box
double height; // Height of a box
```

```
// Member functions declaration
double getVolume();
void setLength( double len );
void setBreadth( double bre );
void setHeight( double hei );
};
// Member functions definitions
double Box::getVolume()
{
    return length * breadth * height;
}
void Box::setLength( double len )
{
    length = len;
}
void Box::setBreadth( double bre )
{
    breadth = bre;
}
void Box::setHeight( double hei )
{
    height = hei;
}
// Main function for the program
int main( )
{
    Box Box1; // Declare Box1 of type Box
    Box Box2; // Declare Box2 of type Box
    double volume = 0.0; // Store the volume of a box here
    // box 1 specification
    Box1.setLength(6.0);
    Box1.setBreadth(7.0);
    Box1.setHeight(5.0);
    // box 2 specification
    Box2.setLength(12.0);
    Box2.setBreadth(13.0);
    Box2.setHeight(10.0);
    // volume of box 1
    volume = Box1.getVolume();
    cout << "Volume of Box1 : " << volume <<endl;
    // volume of box 2
    volume = Box2.getVolume();
    cout << "Volume of Box2 : " << volume <<endl;
    return 0;}
```

3.9. Inheritance

One of the most important concepts in object-oriented programming is that of inheritance. Inheritance allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code functionality and fast implementation time. When creating a class, instead of writing completely new data members and member functions, the programmer can label that the new class should inherit the members of an existing class. This existing class is called the base class, and the new class is referred to as the derived class.

The idea of inheritance implements the is a relationship. For example, mammal IS-A animal, dog IS-A mammal hence dog IS-A animal as well and so on.

3.9.1. Base & Derived Classes

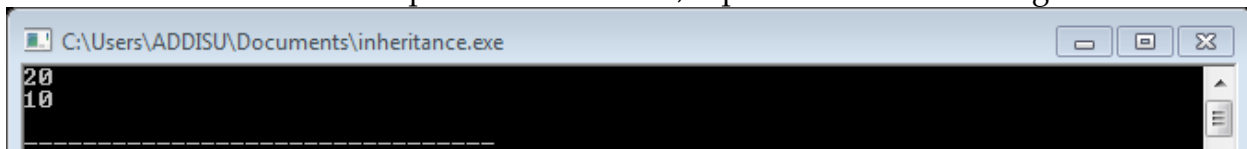
A class can be derived from more than one classes, which means it can inherit data and functions from multiple base classes. To define a derived class, we use a class derivation list to specify the base class(es). A class derivation list names one or more base classes and has the form: class derived-class: access-specifier base-class where access-specifier is one of public, protected, or private, and base-class is the name of a previously defined class. If the access-specifier is not used, then it is private by default.

Consider a base class Polygon and its derived class Rectangle as follows:-

```
// derived classes
#include <iostream>
using namespace std;
class Polygon {
protected:
int width,
int height;
public:
void set_values (int a, int b)
{ width=a;
height=b;
}
};
class Rectangle: public Polygon {
public:
int area ()
{ return (width * height); }
};
class Triangle: public Polygon {
public:
int area ()
{ return (width * height / 2); }
```

```
};  
int main () {  
    Rectangle rect;  
    Triangle trgl;  
    rect.set_values (4,5);  
    trgl.set_values (4,5);  
    cout << rect.area() << endl;  
    cout << trgl.area() << endl;  
    return 0;  
}
```

When the above code is compiled and executed, it produces the following result:



3.9.2. Access Control and Inheritance

A derived class can access all the non-private members of its base class. Thus base-class members that should not be accessible to the member functions of derived classes should be declared private in the base class. We can summarize the different access types according to - who can access them, in the following way:

Access	public	protected	private
Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

3.9.3. Type of Inheritance

When deriving a class from a base class, the base class may be inherited through public, protected or private inheritance. The type of inheritance is specified by the access-specifier as explained above. We hardly use protected or private inheritance, but public inheritance is commonly used. While using different type of inheritance, following rules are applied:

□□Public Inheritance: When deriving a class from a public base class, public members of the base class become public members of the derived class and protected members of the base class

Become protected members of the derived class. A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class.

□□□Protected Inheritance: When deriving from a protected base class, public and protected members of the base class become protected members of the derived class.

- Private Inheritance: When deriving from a private base class, public and protected members of the base class become private members of the derived class.

3.9.4. Multiple Inheritances

A C++ class can inherit members from more than one class and here is the extended syntax:

class derived-class: access baseA, access baseB....

Where access is one of public, protected, or private and would be given for every base class and they will be separated by comma as shown above.